

Graph-Driven Development

*The human owns intent. The agent owns labor. A **graph of dependencies** sits between them as the contract.*

§ 01 Forward motion needs the *graph*.

Every prompt to an AI seeds scope, even weakly. The hard question shows up later — after the agent finishes a unit of work and looks for the next one.

An autonomous agent gets handed a task, does it, opens a PR. Good work. Now what?

PLAIN VERSION

An **autonomous agent** is an AI worker you assign a task to and leave running. It picks its own files, writes its own code, opens its own pull request. Jules, Devin, OpenHands, and similar tools.

Two failure modes show up at that decision point.

Invent. Scope was actually unclear — a client tool, ambiguous priorities, unwritten constraints. The agent fills the gap by guessing what comes next. The new PR is technically fine and conceptually wrong. Loud failure.

Idle. Scope was actually clear — well-defined project, obvious next ten steps, a human who would be perfectly happy with autonomous forward motion. But the agent stops and waits, because nothing told it whether proceeding is safe. Quiet failure — wasted context, momentum, cycles.

*The question isn't who picks scope.
It's when forward motion is safe.*

Think about how a video game gates its world. Large regions of the map start locked. The game requires you to fully explore and prove competence in your current area before the next region unlocks – and when you do, it unlocks automatically. You don't get a popup asking if you're ready. You literally proved it.

GDD is the engineering discipline behind that pattern. The map is a written-down dependency graph. Acceptance criteria are the competence demonstrations. When a node is marked complete, the runtime sees its dependents as ready and dispatches them – automatically, no human typing "go." When the graph is silent or contradictory, the agent waits.

WHAT IS A GRAPH?

A **dependency graph** is a project plan drawn as boxes and arrows. Each box is a unit of work – a **node**. An arrow from A to B means "B can't start until A is done."

The agent doesn't draw the map. It receives one ready node, executes it, reports back. A human reviews completed work, accepts what counts, and writes new nodes when new clarity emerges. Forward motion is autonomous *within* a well-formed graph.

§ 02 Two repositories, one *doctrine*.

The cast: two git repositories,
one external agent, one human
who alone can change the map.

`gddp-config` holds project truth — nodes, schemas, templates.
Nothing runs here. Humans write; agents read. The first is a
repository, which is just a directory tracked by git.

WHAT IS A REPOSITORY?

A **repository**, or "repo," is a directory of files tracked by `git`. Splitting GDD into two repos puts a wall between things humans edit (truth) and things machines edit (state).

`gddp-runtime` is the machinery: a script that wakes on a schedule, reads the graph, finds work, dispatches it, records what comes back. It never edits project truth.

This separation is the whole game. Truth and activity live apart; activity never rewrites truth.

FIG. 2.1 — THE TWO-REPO APPARATUS



The executor is external; the human reviewer is the only writer to truth.

What a "node" looks like on disk

A node is a YAML file. Inside you find what a well-written ticket would have, plus a few things it wouldn't.

YAML, BRIEFLY

YAML is a readable plain-text format for structured data. Indentation matters. Keys end in a colon. Lists start with a dash.

```
node_id: lead-intake
purpose: Capture landscaping lead information and normalize it into
acceptance:
  # the conditions a human will check against
  - form accepts name, contact, project type, budget, location, time
  - submission creates one canonical Lead record
  - downstream estimator receives normalized project_type values
  - no duplicate client record is created
required_artifacts:
  - tests/lead-intake.test.ts
  - artifacts/node-runs/lead-intake/report.md
semantic_contracts:
  outputs: [Lead.created, EstimateInput.ready]
  must_not:
    - write directly to QuickBooks
    - create LMN proposal before consultation status
```

Two things matter. The node is a **contract** — it states what must hold true for the work to count. And the contract is written by a person before any agent reads it. Inheritance, not authorship.

The shape of every node is policed by a schema in the config repo.

WHY A SCHEMA?

A **schema** is the rulebook for what a file is allowed to contain. `gddp-config/schemas/v1/` holds one for every kind of object — node, job, result, event.

NOTE

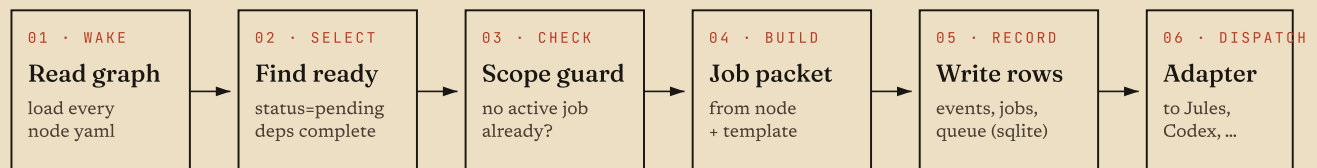
Acceptance vs. tests. *Tests prove the code runs. Acceptance proves the code did what was asked. A function can pass tests while completing the wrong objective.*

§ 03 How a node becomes a *job*.

The forward lane turns a written-down node into a unit of agent labor.

A script runs on a schedule, reads files, writes rows in a database, posts a GitHub issue. That's it.

FIG. 3.1 — ONE TICK OF THE HEARTBEAT



one tick – runs every n seconds, walks left to right, sleeps

Six steps. The loop lives in `scripts/runtime/heartbeat/runner.py`.

At the center is the *heartbeat*. Each tick asks the same questions.

HEARTBEAT, PLAINLY

A **heartbeat** is a script that runs on a timer — say, every five minutes — and does the same checks each time. Cadence is a knob; the rhythm is the point.

What "ready" means

A node is ready when two conditions hold. Its own `status` is `pending`, and every node it *depends on* has `status: complete`. If both are true, it is fair game for dispatch. If either fails, it sits.

DEPENDENCIES, PLAINLY

A node's **dependencies** are the other nodes it lists as prerequisites. If B says "depends on: A," then A must be complete before B can be picked up.

Same logic as `make`. You cannot skip ahead.

What gets dispatched

The runtime builds a *job packet* – purpose, acceptance criteria, required artifacts, target repository, fresh job identifier.

An *adapter* translates the packet into something the chosen executor understands. Today the only working adapter is `jules_action_adapter.py`, which opens a GitHub issue with the `jules` label.

ADAPTER, PLAINLY

An **adapter** makes Agent X think the runtime is talking to it the way Agent X expects. New executor = new adapter, same core machinery.

Adding Codex or a local harness means a new adapter and nothing else.

Origin. *GDD started as a coordination question – how do Jules and OpenClaw work together without stepping on each other? The graph-as-shared-contract answer generalized past those two specific agents, and the adapter abstraction is what made it agent-agnostic. Anything that can read a packet and return a PR can dispatch. Claude's scheduling layer joined later under the same interface.*

Why state goes to a database

Every event, job, queue record, and result is written to a small *SQLite* file – `db/queue.db`.

SQLite, plainly

SQLite is a database whose entire contents live in a single file on disk. No server, no login. You inspect it with `sqlite3` at the terminal.

Reason: auditability. The graph tells you what *should* happen. The database tells you what *did*.

NOTE

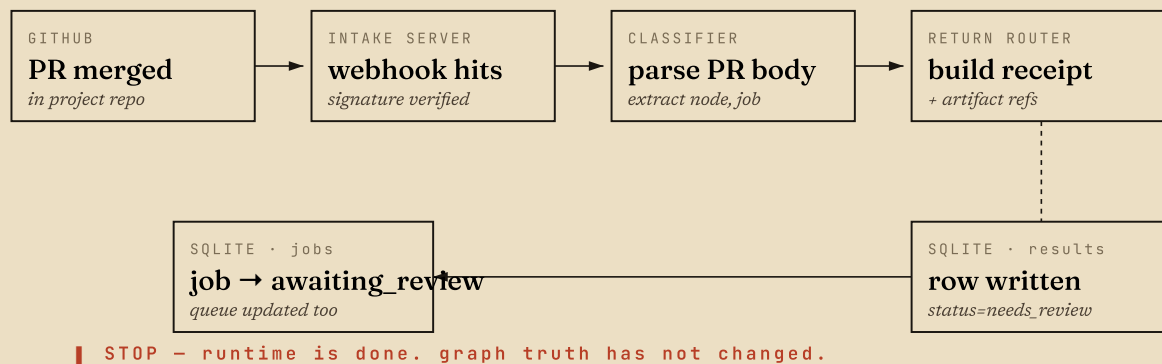
Pull request? *A PR is the standard way to propose changes in git – push a branch, ask to merge. In GDD, every successful job returns as a PR.*

§ 04 How a merged PR becomes a *receipt*.

Forward lane sends work out;
return lane receives evidence back.

This lane stops on purpose, well before declaring
anything complete.

FIG. 4.1 – THE RETURN LANE (RECEIPT-ONLY)



The lane writes a receipt and updates job state. It does not update the graph.

The path, traced

A PR is merged. A webhook hits the intake server and is signature-verified. A classifier parses the PR body and extracts the node and

job identifiers. A return router builds a receipt with artifact references and writes a row to the `results` table.

WEBHOOK, PLAINLY

A **webhook** is a URL that another service (here, GitHub) hits with a JSON payload when something happens (here, a PR merging). Signature verification keeps strangers out.

What lives in a receipt

A receipt is a row in the `results` table. It holds the job and node identifiers, the PR URL, paths to artifacts the agent produced, and a status — always `needs_review` at this stage.

RECEIPT VS. COMPLETION

A **receipt** says "the agent claims it did the work, here is the evidence."

Completion is the output of a review, and only a human produces it.

The receipt is not a verdict. The runtime never makes verdicts. The receipt exists to be readable by the human who will.

*Merging proves the code merged.
Not that it did what was wanted.*

The deliberate stop

An earlier version of this system treated a merged PR as proof of completion and auto-updated the graph. Wrong. Conflating merge with completion is the slip that fails agentic systems over time.

The return lane was severed from graph truth. The runtime stops at `awaiting_review`. Next move belongs to a human. Receipts only; no auto-completion in this phase.

§ 05 Five hands the operator can *play*.

When a job lands in `awaiting_review`, the runtime is finished with it.

The operator opens the PR, the receipt, and the node's acceptance criteria, and chooses exactly one of five named actions. All explicit. The boundary between activity and truth is drawn here.

ACTION

WHAT IT MEANS

accept

The work meets acceptance. The operator edits the node's YAML in `gddp-config` by hand and commits. Only path that moves the graph forward.

retry

Something failed but is salvageable. The job is re-dispatched, often with a sharper instruction.

block

The work cannot move forward right now. Graph truth stays unchanged; blocker is recorded outside the runtime.

defer

Receipt is fine, but acceptance isn't the right move yet – perhaps a sibling node needs to land first.

reopen

Later evidence invalidates earlier work. The node is revisited or replaced; the graph is edited accordingly.

Editing the project graph is the act of saying "I accept this as truth." Removing that gesture would smuggle authority back into the runtime. The friction is the feature.

Why exactly five

The smallest vocabulary that covers every realistic review outcome without smuggling decisions into the runtime. None of these verbs is "the runtime decided." Accept, reject, defer – all belong to a person.

"Ignore" is not on the list. Doing nothing is *defer* – a real decision on a real desk.

*The runtime is allowed to know
what happened. Not what it means.*

Reading a receipt in practice

Open the PR and read the diff. Read the receipt row in `results`. Walk through the node's `acceptance` list, one item at a time, checking each against the PR. Read the required artifacts.

If everything lines up – and only then – open the node's YAML and edit the status to `complete` in a hand-written commit.

Slower than automatic acceptance. That's the point — slowness is where judgment lives, and judgment is the load-bearing thing the agent cannot supply.

NOTE

Known limit. *The current return router has a hardcoded allowlist for `skchaudr/vault-doctor`. A scaling step is to lift that into config.*

§ 06 You learn the boundary by *practicing* it.

Reading tells you a system's shape. Practicing tells you what it feels like to be wrong about it.

Do these at a real terminal on a small real project. Answers come from files, rows, and receipts – not intuition. They're sequenced; later ones depend on earlier ones.

01 Synthetic Receipt Drill

Run `python3 scripts/init_db.py`, then `python3 scripts/dry_run.py`. No real repo, no real agent. Inspect `events`, `jobs`, and `results`. Confirm the synthetic job ends in `awaiting_review` and no graph file changed.

02 Forward Dispatch, Tiny Real Repo

Add a ready node to `gddp-config` for a low-stakes side project – a `--dry-run` flag, a missing unit test, a healthcheck endpoint. Run the heartbeat. Explain in words why *that* node was chosen and what was inside the packet.

03 Receipt From a Merged PR

Take the Run 2 job to completion: the agent opens a PR, you merge, the webhook fires, the receipt lands. Inspect the `results` row. Confirm receipt is `needs_review`, job is `awaiting_review`, no file in `gddp-config` changed.

04 Human Accept Drill

Walk through the acceptance criteria, one item at a time, against the merged PR. If everything matches, open the YAML and update status to `complete` by hand. Commit. Write down: what evidence made you comfortable accepting?

05 Human Reject or Defer Drill

Run the full lane with a PR that is intentionally incomplete. Receive the receipt, then choose one of *retry*, *block*, *defer*, or *reopen*. Do not touch the graph. Write down the exact condition that blocked acceptance.

Questions every run must answer

§ *What created this job?*

§ *Why was this node selected?*

§ *What exactly was dispatched?*

§ *What did the executor return?*

§ *Where is the receipt?*

§ *Why is this job in `awaiting_review`?*

§ *What exact human action would change graph truth next?*

If you can't answer all seven from files and rows — not memory — the drill isn't done. Repeat with a different tiny task. A fuzzy mental model becomes a system you can read.

§ 07 From local correctness to *systemic meaning*.

The next phase makes the graph
not just the scheduler of work
but the validator of meaning.

Treat this as a road map, not a release.

The pivot: unit tests prove a function behaves. Acceptance proves a node delivered. Neither proves the project is still coherent.

UNIT TEST, PLAINLY

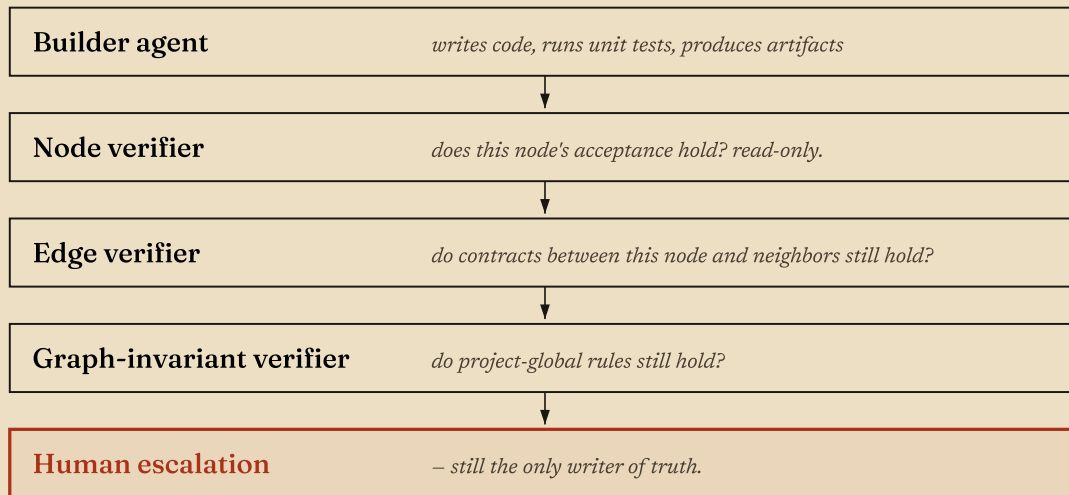
A **unit test** is a small piece of code that checks one function does what it claims. The cheapest signal that a single brick is square. It says nothing about the wall.

A node can pass its acceptance while quietly violating a contract that a downstream node was counting on. The next-generation runtime adds a stack of verifiers whose only job is to notice this drift.

CONTRACT, PLAINLY

A **contract** between two nodes is the promise the upstream one makes about its outputs and the downstream one makes about its inputs. When written down, an edge verifier can check it.

FIG. 7.1 – THE VERIFIER HIERARCHY (NEXT PHASE)



Each layer returns pass, fail, or uncertain. None edit code.

Why one verifier is not enough

A node adds a "send invoice" button. The builder writes it. Unit tests pass. Acceptance says "user can send an invoice"; it passes. But two nodes upstream, the graph promised an invariant: *no direct writes to QuickBooks before consultation*.

INVARIANT, PLAINLY

An **invariant** is a rule always supposed to be true, no matter what changes. "Every user has one primary email." Invariants are what make a system feel coherent over time.

The button writes directly to QuickBooks. A node verifier clears it. Only an edge or graph-invariant verifier notices. Traditional pipelines prove component correctness. GDD attempts *systemic correctness*.

SEMANTIC CORRECTNESS, PLAINLY

Semantic correctness means the system did what it meant to do, not just what it was technically asked to do. A retrieval system returning "a discussion of the truth" instead of "the truth" is semantically incorrect – and passes every unit test.

The scaling stages

STAGE 1

Single node, single agent

Where the system is now. One node picked manually. One agent. The operator runs the drills against it.

STAGE 2

Multiple agents, one graph

Independent ready nodes dispatched in parallel. The graph says who owns which path.

STAGE 3

Verifier agent added

A read-only agent reviews the diff, tests, and spec. Returns pass / fail / uncertain. Cannot edit code.

STAGE 4

Dispatcher expansion

A scheduler reads the graph, opens panes in a multi-pane terminal like CMUX, injects prompts, watches reports, updates state.

STAGE 5

Semantic graph gate

The project cannot be called done until a graph-level verifier confirms every completed node still satisfies upstream and downstream contracts.

BEYOND

OpenClaw & CI for meaning

A reasoning layer that surfaces uncertainty earlier and more legibly. CI/CD where the build target is a graph of meaning.

CMUX, BRIEFLY

CMUX is a multi-pane terminal built for running several agents side by side. In the next-phase sketch, it is the cockpit the dispatcher uses. The graph remains the source of truth.

Builders build. Verifiers verify. The graph decides what done means.

The non-negotiable thing

One rule doesn't move across stages: **the agent doesn't decide if its own work passes**. Verifiers make the human judgment cheaper and faster — they don't replace it.

GRAPH-DRIVEN DEVELOPMENT • FIELD MANUAL • V1 • 2026